

**TCS3351 CRYPTOGRAPHY AND DATA SECURITY**  
**Tutorial 10 - Cryptographic Hash Functions/Message Authentication**  
**Codes**

1. Briefly explain how the hash function can be used for message authentication without the usage of encryption function.

Answer:

When sender wishes to send a message  $M$ , he will compute the message digest  $h = H(M)$ , sends the pair of  $\{M, h\}$  to receiver.

Receiver when receiving the pair  $\{M, h\}$ , he checks whether  $H(M) = h$ . If it matches, then the  $M$  is valid.

Note: Provided there is no man in the middle attack or no impersonation.

```
D:\openssl\aa>echo Message_Authentication > M.txt

D:\openssl\aa>openssl md5 M.txt
MD5(M.txt)= 3cc8145a3b6034e2cc61e694b9dd9e94

D:\openssl\aa>openssl md5 M.txt > M.hash

D:\openssl\aa>cp M.txt M.Sent

D:\openssl\aa>cat M.Sent
Message_Authentication

D:\openssl\aa>type M.hash >> M.Sent

D:\openssl\aa>cat M.Sent
Message_Authentication
MD5(M.txt)= 3cc8145a3b6034e2cc61e694b9dd9e94

// Bob received the M.Sent and start processing!

D:\openssl\aa>head -n 1 M.Sent > M.Receive

D:\openssl\aa>tail -n 1 M.Sent > M.HashReceive

D:\openssl\aa>openssl md5 M.Receive
MD5(M.Receive)= 3cc8145a3b6034e2cc61e694b9dd9e94

D:\openssl\aa>cat M.HashReceive
MD5(M.txt)= 3cc8145a3b6034e2cc61e694b9dd9e94

D:\openssl\aa>
```

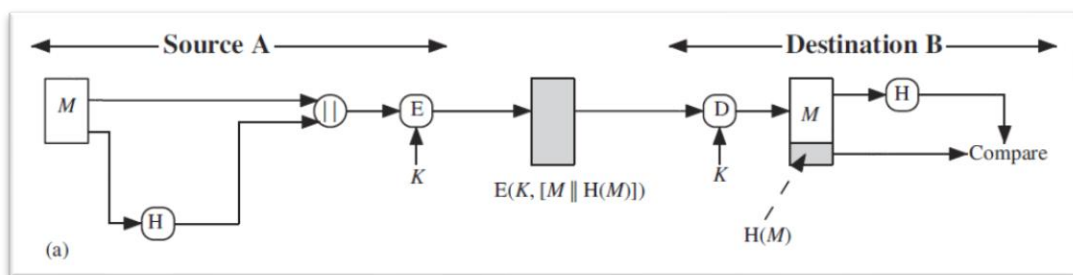
2. Briefly explain the man-in-the-middle attack/ impersonation (Someone pretend to be the sender) on the hash function and how it can be prevented.

Answer:

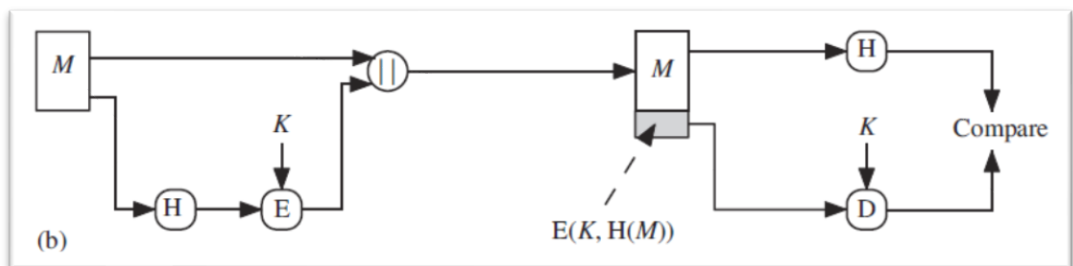
In man-in-the-middle attack, an attacker intercepts the sent message  $M$  and the message digest (hash value)  $h = H(M)$ . He can then replace  $M$  with his chosen message  $M'$  along with the computed message digest  $h'$ , transmits it as  $\{M', h'\}$ . When the receiver receives  $\{M', h'\}$ , he will not be able to distinguish if the original message  $M$  has been altered since he will always correctly verify the message digest  $H(M') = h'$ . Hash function is std and there is no secret information to protect anything!

To prevent such an attack, several methods can be used.

- a) **Method 1:** The message  $M$  and its message digest  $h = H(M)$  is concatenated and encrypted further using a symmetric cryptosystem. That is,  $C = \text{Enc}(K, M \parallel h)$ . The receiver will firstly decrypt the ciphertext to obtain message  $M$ , and next check its authenticity by verifying its message digest  $H(M) = h$ .

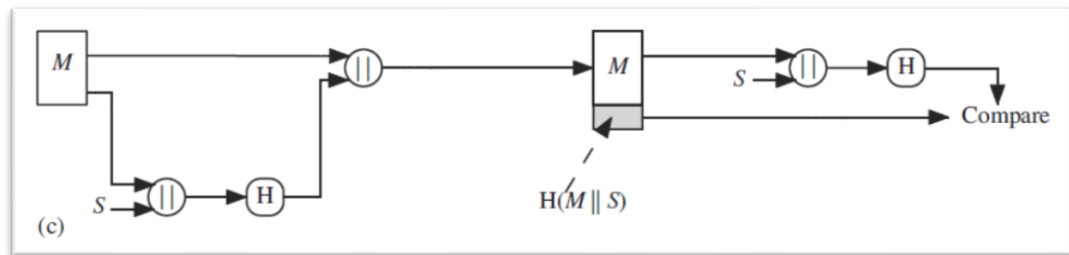


- b) **Method 2:** Only the message digest  $h = H(M)$  is encrypted using a symmetric cryptosystem, and then sent along with the message to the receiver as  $\{M, \text{Enc}(K, h)\}$ . The receiver will decrypt the encrypted hash value  $h$ , and next verify its authenticity whether the computation  $H(M) = h$ .



- c) **Method 3:** The message digest  $h = H(M \parallel S)$  is computed using concatenation of the message  $M$  and a pre-established secret value  $S$  between sender and receiver, and then sent along with the message to the receiver as  $\{M, h\}$ . The receiver will verify the

authenticity of the message by checking whether the computation  $H(M \parallel S) = h$ . The attacker does not have  $S$ , so he can never produce the correct  $h$ .



```
D:\openssl\aa>echo Message_Authentication > M.txt
D:\openssl\aa>echo Password=abc123 > P
D:\openssl\aa>cp M M.Process
D:\openssl\aa>cat P >> M.Process
D:\openssl\aa>cat M.Process
    Message_Authentication
    Password=abc123

D:\openssl\aa>openssl md5 M.Process > M.hash
D:\openssl\aa>cat M.hash
MD5(M.Process)= 2737c8c7027bb846427312548d6bde81
D:\openssl\aa>cp M M.sent

D:\openssl\aa>cat M.sent
    Message_Authentication

D:\openssl\aa>cat M.hash >> M.sent
D:\openssl\aa>cat M.sent
Message_Authentication
MD5(M.Process)= 2737c8c7027bb846427312548d6bde81

//Send the M.Sent to Bob.
```

// Bob Action.

```
Message_Authentication
MD5(M.Process)= 2737c8c7027bb846427312548d6bde81

D:\openssl\aa>head -n 1 M.sent > M1

D:\openssl\aa>tail -n 1 M.sent > M1.hash

D:\openssl\aa>cp M1 M1.1
D:\openssl\aa>cat P >> M1
D:\openssl\aa>mv M1.1 M1.bak
D:\openssl\aa>cat M1
Message_Authentication
Password=abc123

D:\openssl\aa>openssl md5 M1
```

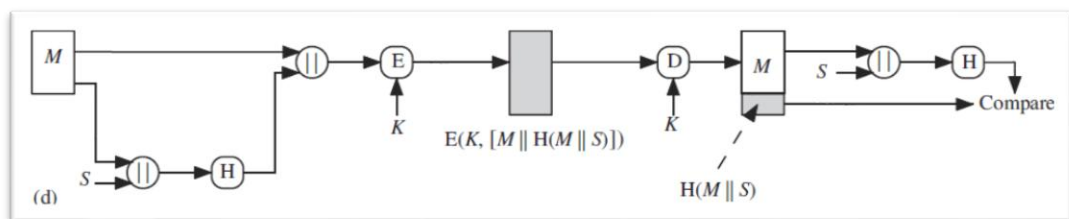
```
MD5(M1)= 2737c8c7027bb846427312548d6bde81
```

```
D:\openssl\aa>cat M.hash
```

```
MD5(M.Process)= 2737c8c7027bb846427312548d6bde81
```

```
D:\openssl\aa>
```

- d) **Method 4:** The message digest  $h = H(M \parallel S)$  is computed using concatenation of the message  $M$  and a pre-established secret value  $S$  between sender and receiver. This message digest is concatenated with message  $M$  and further encrypted using a symmetric cryptosystem. That is  $C = \text{Enc}(K, M \parallel h)$ . The receiver will firstly decrypt the ciphertext  $C$ , extract the message  $M$ , and finally verify its authenticity by checking whether the computation  $H(M \parallel S) = h$ .



3. State the difference between weak and strong collision resistance in the case of cryptographic hash functions.

Answer:

**Weak collision resistance:** Also known as second pre-image resistance, for any given  $x$  and its corresponding hash  $H(x)$ , it is computationally infeasible to find another  $y \neq x$  such that  $H(y) = H(x)$ . E.g sha256 for Win11.iso, Win11+virus.iso that have the same hash value.

**Strong collision resistance:** It is computationally infeasible to find a pair of  $(x, y)$  with  $x \neq y$  such that  $H(x) = H(y)$ . Nothing is known upfront!

The weak collision resistance explains the infeasibility to search for another element  $y$  to obtain the identical hash value, given initially that the block  $x$  and its corresponding  $H(x)$  are known. Different from the strong collision resistance, where one finds arbitrary pair of  $(x, y)$  block that will produce the same hash value of  $H(x) = H(y)$ .

4. Briefly explain the model in which the public-key encryption can be used for confidentiality, authentication, and digital signature

Answer:

Alice 1 2 send a signed large document to Bob and only Bob can see the document. (confidentiality, authentication and non repudiation).

- Alice
  - 1. Generate a random password. (key distribution)
  - 2. Encrypt(RSA, password, BobPub) => CipherPassword

- 3.  $\text{Encrypt}(\text{DES}, \text{LargeDoc}, \text{password}) \Rightarrow \text{CipherTxt}$
- 4.  $\text{Hash}(\text{SHA256}, \text{LargeDoc}) \Rightarrow \text{dochash}$
- 5.  $\text{Sign}(\text{dochash}, \text{AlicePriv}) = \text{signature}$
- 6. Send signature, CipherTxt, CipherPassword to Bob.
- Bob
  - 1.  $\text{Verify}(\text{Signature}, \text{AlicePub}) = \text{True/False}$ .
  - 2. If true, then proceed. Then you will get the dochash. Then, Bob  $\text{Decrypt}(\text{RSA}, \text{CipherPassword}, \text{BobPriv}) \Rightarrow \text{password}$
  - 3.  $\text{Decrypt}(\text{DES}, \text{CipherTxt}, \text{password}) \Rightarrow \text{LargeDoc}$
  - 4.  $\text{Hash}(\text{SHA256}, \text{LargeDoc}) = \text{dochash}$  (verify that the doc is not changed)

If you can do it in openssl!. Encrypt and decrypt, man in the middle cannot get the real msg.

5. Briefly explain how the message authentication and confidentiality can be provided by Message Authentication Code (MAC) with authentication tied to plaintext and how it is different from authentication tied to ciphertext.

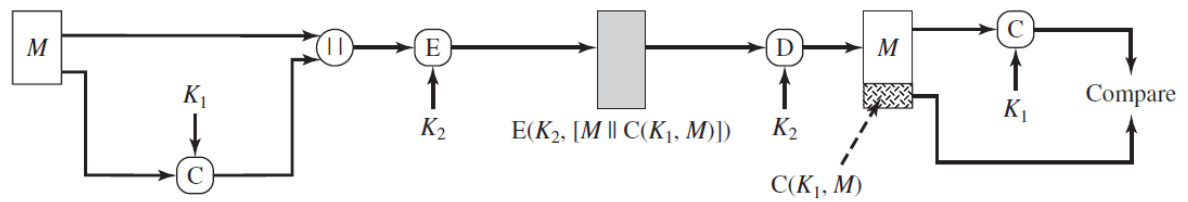
Answer:

C = Cryptographic Checksum function. Integrity.

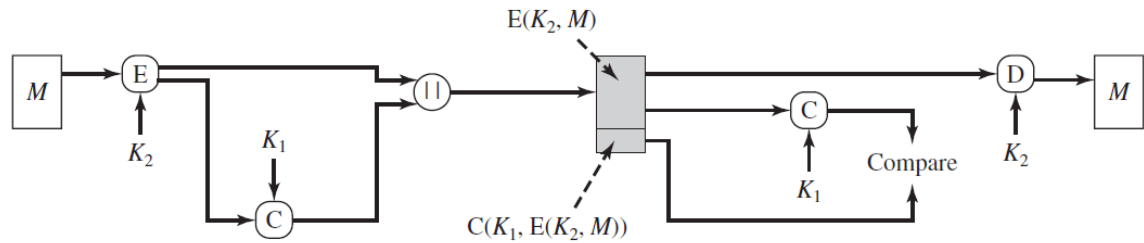
E = Encryption. (confidentiality) D= Decryption.

In MAC with authentication tied to plaintext, the MAC is firstly computed using MAC function with message  $M$  and secret key  $K_1$ ,  $\text{MAC} = C(K_1, M)$ . This MAC value is then concatenated with the message for the next encryption using another symmetric key  $K_2$  to produce  $\text{Ciphertext} = E_{K_2}(M \parallel \text{MAC})$ . On the receiver side, he firstly decrypts the Ciphertext using symmetric key  $K_2$  to recover  $(M \parallel \text{MAC})$ , extract  $M$  and next compute MAC using secret  $K_1$ ,  $\text{MAC}' = C(K_1, M)$ . The computed  $\text{MAC}'$  is compared with MAC received. If there is a match, the message is accepted as valid, otherwise it is rejected.

In MAC with authentication tied to ciphertext on the other hand, the message  $M$  is firstly encrypted using symmetric key  $K_2$  to produce  $\text{Ciphertext} = E_{K_2}(M)$ , and next to compute MAC from the ciphertext using secret key  $K_1$ ,  $\text{MAC} = C(K_1, \text{Ciphertext})$ . The MAC is then concatenated with the Ciphertext as  $(\text{Ciphertext} \parallel \text{MAC})$  and transmitted over to the receiver. On the receiver side, he firstly extracts Ciphertext from  $(\text{Ciphertext} \parallel \text{MAC})$  and decrypt it using symmetric key  $K_2$  to recover the message  $M$ . Meanwhile, MAC is computed from the Ciphertext using secret key  $K_1$ ,  $\text{MAC}' = C(K_1, \text{Ciphertext})$ . The computed  $\text{MAC}'$  is compared with MAC received. If there is a match, the message is accepted as valid, otherwise it is rejected.



(b) Message authentication and confidentiality; authentication tied to plaintext



(c) Message authentication and confidentiality; authentication tied to ciphertext

### References:

1. William Stallings, "Cryptography and Network Security – Principles and Practice", Seventh Edition, Prentice Hall, 2017
2. Forouzan, B. A. & Mukhopadhyay, D. (2015). Cryptography & Network Security (3rd ed.). McGraw-Hill.